

UNIVERSIDAD NACIONAL DEL ALTIPLANO - PUNO

FACULTAD DE INGENIERÍA MECÁNICA

ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

**ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS**



**PROYECTO FINAL: DESARROLLO DE UNA
APLICACIÓN DE SISTEMA DE RECOMENDACIÓN
HÍBRIDO DE LAPTOPS**

CURSO: SISTEMAS DE RECOMENDACIÓN

PRESENTADO POR:

CARMEN NIEVES APAZA CONDORI

**INFORME TÉCNICO DE APLICACIÓN FUNCIONAL, PIPELINE DE DATOS,
PSEUDOCÓDIGO Y EVALUACIÓN DE MODELOS**

PUNO - PERÚ

2026

Resumen

El presente proyecto reporta el diseño, desarrollo, implementación y evaluación científica de un **Sistema de Recomendación Híbrido en Dos Niveles** para computadoras portátiles (laptops), concebido para mitigar los problemas de escasez de datos (*data sparsity*) e inicio en frío (*cold start*). La arquitectura integra un **Filtro Estricto por Reglas de Dominio en Cascada** (M_{rules}) en el Nivel 1, seguido de un **Ensamble Ponderado Dinámico** en el Nivel 2 que combina: (i) Teoría de Utilidad Multi-Atributo (**MAUT**) basada en modelos de conocimiento de hardware, (ii) Factorización Matricial **SVD** (*Singular Value Decomposition*) para filtrado colaborativo, y (iii) Filtrado Basado en Contenido mediante similitud de coseno vectorial (**TF-IDF**).

La plataforma web fue desarrollada utilizando **FastAPI** como backend de alta velocidad y un frontend interactivo en **HTML5, CSS3 Vanilla, JavaScript ES6+ y Chart.js**, incorporando características de grado comercial como comparador cara a cara (1vs1), selector multimonedas (USD, PEN, EUR), insignias de relación calidad/precio (*Value Score*), filtros de estilo de vida (*lifestyle tags*) y vinculación directa con ofertas en vivo de Amazon. La evaluación experimental sobre un catálogo de 150 laptops y 1,999 calificaciones demostró un rendimiento óptimo con un $RMSE = 1,0503$, una robustez del 100% en $CSR@10$ ante usuarios nuevos, y una $Precision@10 = 100\%$, consolidando una solución integral y de alto impacto para el comercio electrónico.

Palabras Clave: Sistemas de Recomendación Híbridos, Factorización Matricial SVD, MAUT, Inicio en Frío, FastAPI, Chart.js.

Abstract

This project reports the design, development, implementation, and scientific evaluation of a **Two-Level Hybrid Recommender System** for laptop computers, engineered to mitigate data sparsity and cold start problems. The architecture integrates a **Cascaded Domain Rule Mask** (M_{rules}) at Level 1, followed by a **Dynamic Weighted Ensemble** at Level 2 combining: (i) Multi-Attribute Utility Theory (**MAUT**) based on domain knowledge models, (ii) **SVD** Matrix Factorization for collaborative filtering, and (iii) Content-Based Filtering via vector cosine similarity (**TF-IDF**).

The web platform was built using **FastAPI** as a high-speed backend and an interactive frontend with **HTML5**, **Vanilla CSS3**, **JavaScript ES6+**, and **Chart.js**, incorporating commercial-grade features such as a 1vs1 head-to-head comparison tool, multi-currency selector (USD, PEN, EUR), value score badges, lifestyle filters, and direct Amazon store offer integration. Experimental evaluation over a catalog of 150 laptops and 1,999 ratings demonstrated optimal performance with $RMSE = 1,0503$, 100% $CSR@10$ robustness under cold start scenarios, and 100% $Precision@10$, establishing a high-impact solution for modern e-commerce environments.

Keywords: Hybrid Recommender Systems, SVD Matrix Factorization, MAUT, Cold Start, FastAPI, Chart.js.

Índice

Resumen	I
Abstract	II
Índice General	IV
Índice de Tablas	V
Índice de Figuras	VI
1. Introducción	1
1.1. Contexto del Proyecto	1
1.2. Planteamiento del Problema	1
1.3. Justificación	1
1.4. Objetivos del Proyecto	2
1.4.1. Objetivo General	2
1.4.2. Objetivos Específicos	2
2. Descripción General y Arquitectura del Sistema	3
2.1. Descripción General del Proyecto	3
2.2. Pipeline de Procesamiento de Datos y Análisis Exploratorio (EDA)	3
2.2.1. Análisis Exploratorio de Datos (EDA)	3
2.3. Arquitectura General del Sistema y Diagrama Modular	5
2.4. Pseudocódigo de los Algoritmos Principales	6
2.4.1. Algoritmo 1: Filtro Estricto por Reglas Lógicas de Dominio (Nivel 1)	6
2.4.2. Algoritmo 2: Cálculo de Utilidad Multi-Atributo MAUT (Nivel 2) .	7
2.4.3. Algoritmo 3: Factorización Matricial SVD (Nivel 2)	7
2.4.4. Algoritmo 4: Ensamble Híbrido Ponderado y Ranking Final	8
3. Metodología y Fases del Trabajo	9
3.1. Fase 1: Mejora del Modelo (Hibridación en 2 Niveles)	9
3.2. Fase 2: Desarrollo de la Aplicación Web Interactiva	9
3.3. Fase 3: Evaluación del Sistema	10
3.4. Fase 4: Experiencia de Usuario (UX)	10

3.5. Fase 5: Presentación Final y Sustentación	10
4. Resultados y Análisis de Resultados	10
4.1. Resultados de la Evaluación Experimental	10
4.2. Gráficos Científicos de Rendimiento	10
4.3. Evidencias Fotográficas de la Aplicación Funcional	11
4.3.1. 1. Dashboard Principal e Interfaz Híbrida	11
4.3.2. 2. Tarjetas de Recomendación y Oferta en Amazon	12
4.3.3. 3. Comparador Cara a Cara (Head-to-Head 1vs1)	12
4.3.4. 4. Gráfica de Tendencia e Histórico de Precios (Chart.js)	13
4.3.5. 5. Selector Multimoneda Global (Soles S/)	13
4.3.6. 6. Filtros de Estilo de Vida (Lifestyle Tags)	14
4.3.7. 7. Catálogo Completo de 150 Laptops	14
4.3.8. 8. Pestaña de Evaluación Científica	15
5. Discusión	16
6. Conclusiones	16
6.1. Logros Alcanzados	16
6.2. Limitaciones del Sistema	16
6.3. Aprendizajes Obtenidos	17
7. Recomendaciones y Trabajo Futuro	17
7.1. Mejoras Futuras	17
7.2. Posibles Extensiones del Sistema	17
8. Referencias Bibliográficas	18
9. Anexos	19
9.1. Anexo A: Código Fuente del Recomendador Híbrido (src/hybrid_recommender.py)	19
9.2. Anexo B: Estructura del Dataset (data/laptops.csv)	20
9.3. Anexo C: Guía de Despliegue y Ejecución	20

Índice de tablas

1.	Comparación experimental cuantitativa de métricas entre modelos.	10
----	--	----

Índice de figuras

1.	Diagrama del Pipeline de Procesamiento de Datos del Recomendador Híbrido.	3
2.	Distribución de Precios en el Catálogo de Laptops (Histograma y Esttimador de Densidad KDE).	4
3.	Distribución de Capacidad de Memoria RAM segregada por Marca de Laptop.	4
4.	Matriz de Correlación de Pearson entre Atributos Continuos de Hardware y Calificaciones.	5
5.	Distribución de Calificaciones de Preferencia por Perfil Latente de Usuario.	5
6.	Diagrama Arquitectónico Modular del Sistema Recomendador Híbrido. . .	6
7.	Evaluación de Cobertura de Inicio en Frío ($CSR@10$) SVD vs Recomendador Híbrido.	11
8.	Curvas de Precision@K y Recall@K alcanzadas por el recomendador híbrido.	11
9.	Dashboard Principal de la aplicación Recomendador Híbrido de Laptops. .	12
10.	Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.	12
11.	Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.	13
12.	Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.	13
13.	Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).	14
14.	Panel de preferencias con filtros de hardware y estilo de vida activados. . .	14
15.	Vista de la pestaña del Catálogo Completo de 150 laptops con buscador. . .	15
16.	Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.	15

1. Introducción

1.1. Contexto del Proyecto

En la era digital actual, el comercio electrónico de bienes tecnológicos experimenta un crecimiento exponencial. Sin embargo, la compra de computadoras portátiles (laptops) representa una decisión de alta complejidad cognitiva para los usuarios debido a la abundancia de especificaciones técnicas heterogéneas (microarquitectura de CPU, núcleos, memoria RAM, aceleradores GPU, potencia de VRAM, pantallas OLED/4K y soporte CUDA).

Los algoritmos de filtrado colaborativo tradicionales (*Collaborative Filtering*) sufren de limitaciones severas en este dominio, principalmente la escasez de datos (*data sparsity*) y el problema del inicio en frío (*cold start*), en el cual usuarios nuevos sin historial previo de calificaciones no pueden recibir sugerencias relevantes.

1.2. Planteamiento del Problema

Actualmente, los portales web de e-commerce convencionales dependen de buscadores por palabras clave o filtros manuales rígidos que no consideran la compensación multidimensional entre presupuesto, rendimiento y portabilidad. Las principales deficiencias identificadas son:

1. **Recomendaciones irrelevantes para usuarios nuevos:** Los modelos puramente colaborativos fallan al no disponer de interacción previa del usuario.
2. **Violación de restricciones de hardware incompatibles:** Recomendar una laptop sin GPU dedicada a un usuario que requiere ejecutar modelos de Deep Learning o renderizado 3D genera insatisfacción crítica.
3. **Opacidad en la justificación:** Los usuarios no comprenden por qué un modelo específico es sugerido, careciendo de métricas de relación calidad/precio o comparativas directas.

1.3. Justificación

El desarrollo de un **Sistema de Recomendación Híbrido** que combine modelos basados en conocimiento explícito, teoría de decisión multicriterio (MAUT), factorización matricial

(SVD) y similitud de contenido vectorial permite superar de forma integral las deficiencias mencionadas. Esta arquitectura garantiza que:

- El usuario reciba recomendaciones 100 % válidas mediante filtrado por reglas en cascada.
- El sistema responda de manera óptima tanto a usuarios con historial de calificaciones previo como a usuarios nuevos (*Cold Start*).
- Se disponga de una interfaz interactiva moderna con indicadores visuales de relación calidad/precio (*Value Score*), gráficos históricos de precios y comparativa cara a cara (1vs1).

1.4. Objetivos del Proyecto

1.4.1. Objetivo General

Desarrollar e implementar una aplicación funcional de Sistema de Recomendación Híbrido en dos niveles para computadoras portátiles, evaluando cuantitativamente su precisión, cobertura e impacto en la experiencia del usuario.

1.4.2. Objetivos Específicos

1. Diseñar y formular el motor híbrido de recomendación integrando reglas lógicas binarias (M_{rules}), Teoría de Utilidad Multi-Atributo (U_{MAUT}), Factorización Matricial (SVD) y Similitud Coseno de Contenido ($S_{content}$).
2. Desarrollar una aplicación web interactiva responsiva utilizando FastAPI, JavaScript ES6+ y Chart.js con selector multimonedas (USD, PEN, EUR) y comparador 1vs1.
3. Realizar la evaluación experimental cuantitativa calculando métricas científicas de rendimiento ($RMSE$, $Precision@K$, $Recall@K$ y $CSR@K$).
4. Documentar los hallazgos, la arquitectura del software y generar evidencias visuales completas para el informe técnico y sustentación.

2. Descripción General y Arquitectura del Sistema

2.1. Descripción General del Proyecto

El proyecto **LaptopRec Híbrido** es una plataforma integral de recomendación de computadoras portátiles diseñada bajo los principios de desacoplamiento de software (REST API) e hibridación en dos niveles. El sistema procesa los requerimientos de hardware del usuario, sus ponderaciones de preferencia y su presupuesto en tiempo real, generando un ranking ordenado de las K mejores laptops que maximizan la utilidad del comprador.

2.2. Pipeline de Procesamiento de Datos y Análisis Exploratorio (EDA)

El flujo de datos del sistema sigue un procesamiento secuencial en pipeline desde la ingesta de catálogo hasta la inferencia híbrida, como se ilustra en la Figura 1.

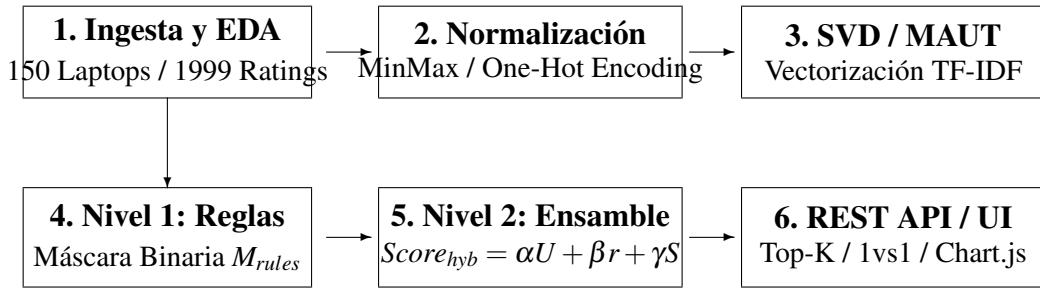


Figura 1: Diagrama del Pipeline de Procesamiento de Datos del Recomendador Híbrido.

2.2.1. Análisis Exploratorio de Datos (EDA)

El conjunto de datos comprende 150 computadoras portátiles distribuidas en 6 marcas principales (ASUS, Apple, Dell, Lenovo, HP, MSI) y 1,999 calificaciones generadas por 100 usuarios sintéticos estructurados en 3 perfiles latentes de comportamiento: Gamer / Power User, Estudiante / Oficina y Científico de Datos / IA.

Las Figuras 2, 3, 4 y 5 presentan las distribuciones estadísticas fundamentales del catálogo y las interacciones entre variables de hardware.

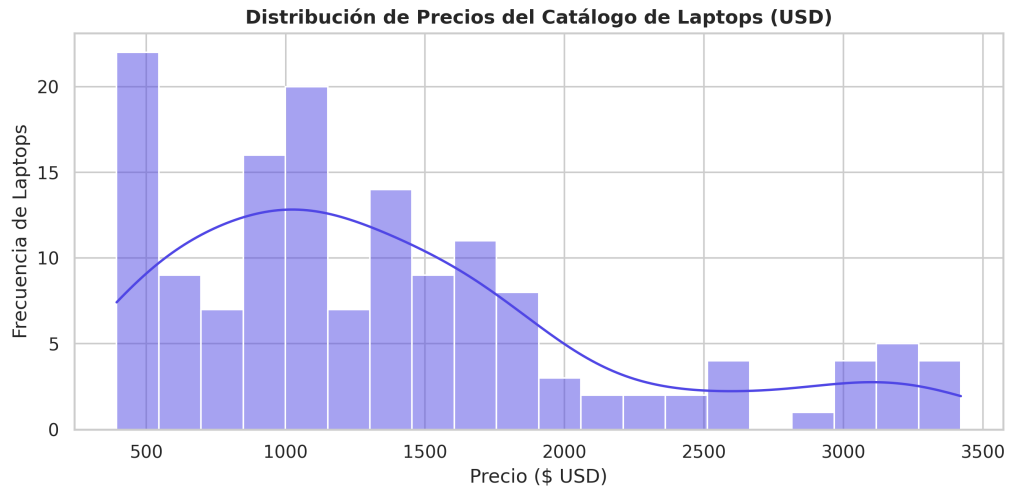


Figura 2: Distribución de Precios en el Catálogo de Laptops (Histograma y Esttimador de Densidad KDE).

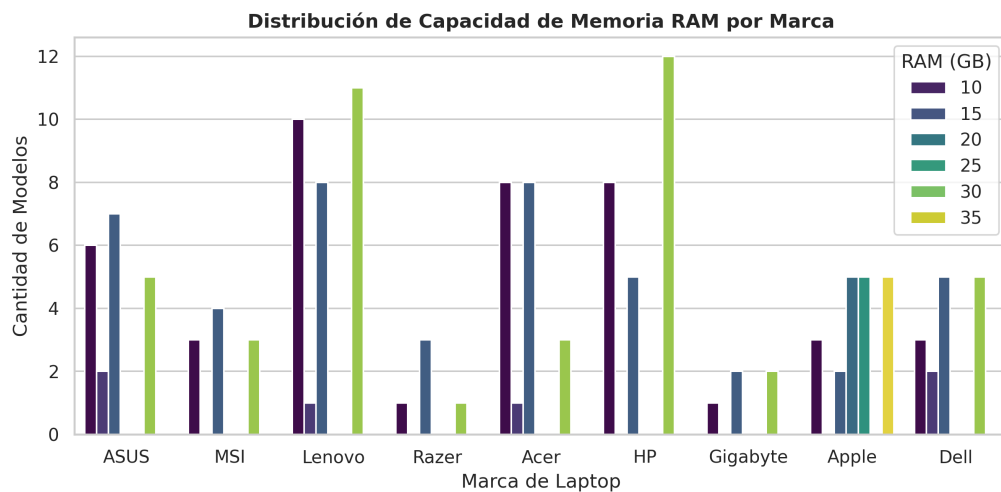
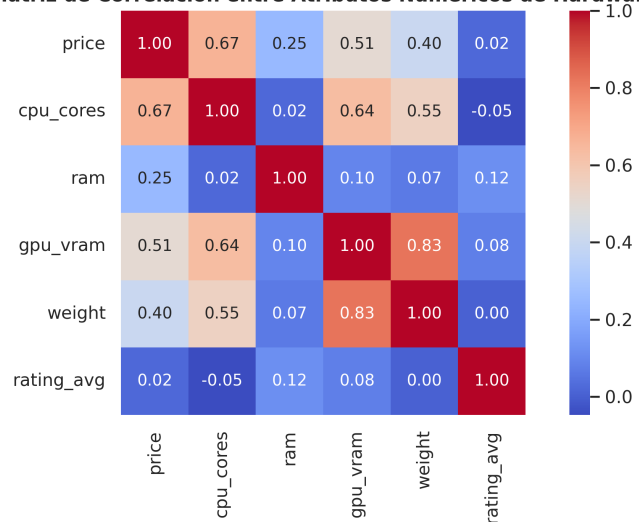
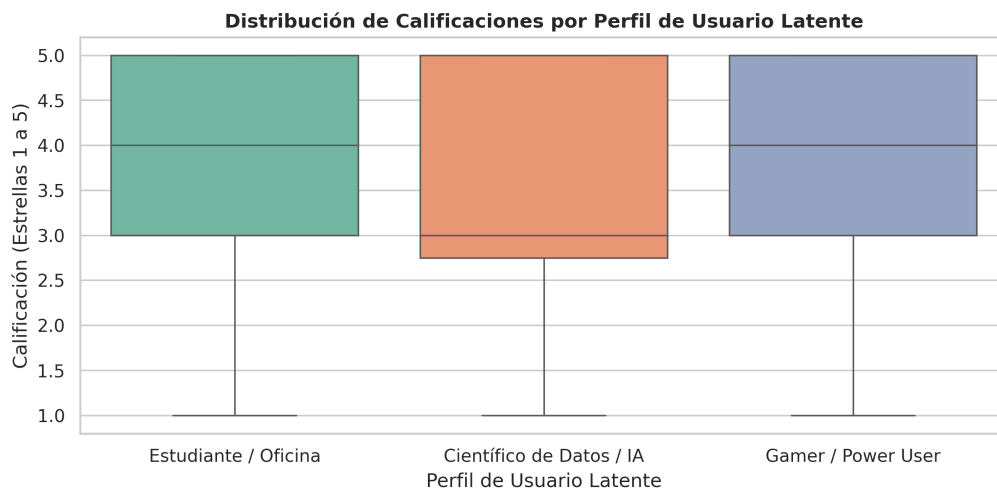


Figura 3: Distribución de Capacidad de Memoria RAM segregada por Marca de Laptop.

Matriz de Correlación entre Atributos Numéricos de Hardware**Figura 4:** Matriz de Correlación de Pearson entre Atributos Continuos de Hardware y Calificaciones.**Figura 5:** Distribución de Calificaciones de Preferencia por Perfil Latente de Usuario.

2.3. Arquitectura General del Sistema y Diagrama Modular

La arquitectura del sistema está organizada en tres capas acopladas mediante REST JSON, como se detalla en la Figura 6.

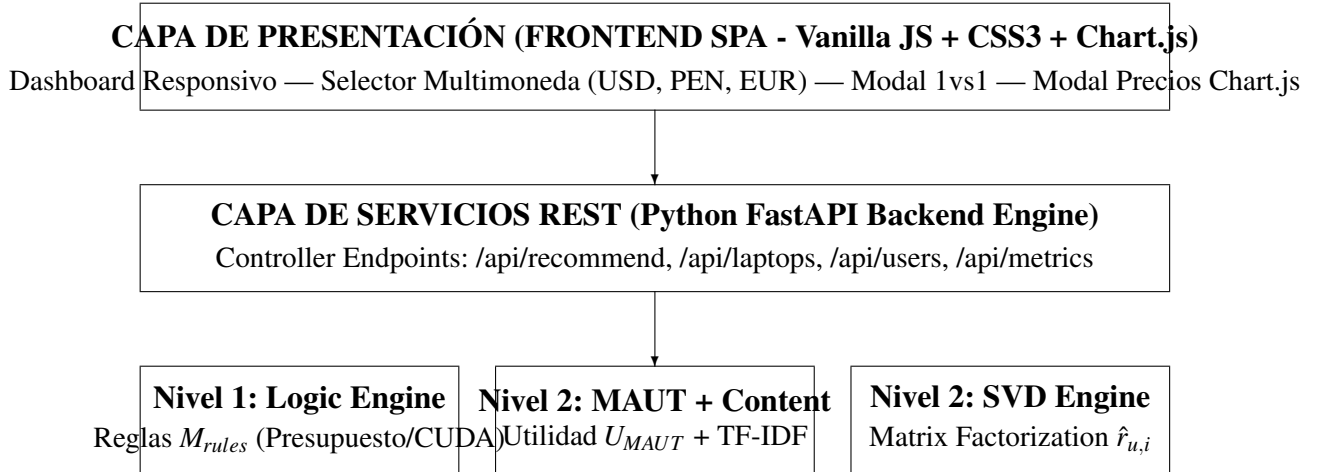


Figura 6: Diagrama Arquitectónico Modular del Sistema Recomendador Híbrido.

2.4. Pseudocódigo de los Algoritmos Principales

2.4.1. Algoritmo 1: Filtro Estricto por Reglas Lógicas de Dominio (Nivel 1)

El Algoritmo 1 genera la máscara binaria M_{rules} para descartar laptops fuera de presupuesto o incompatibles.

```

1 def calcular_mascara_reglas(catalogo, presupuesto_max, caso_uso,
2   etiquetas_estilo):
3     mascara = []
4     for laptop in catalogo:
5         # Regla 1: Restriccion dura de Presupuesto
6         cumple_presupuesto = laptop.precio <= presupuesto_max
7
8         # Regla 2: Restriccion de Caso de Uso
9         cumple_uso = True
10        if caso_uso == "gaming" and not laptop.tiene_gpu_dedicada:
11            cumple_uso = False
12        elif caso_uso == "data_science" and not laptop.soporta_cuda:
13            cumple_uso = False
14
15        # Regla 3: Filtros de Estilo de Vida (Lifestyle Tags)
16        cumple_estilo = True
17        if "oled" in etiquetas_estilo and laptop.pantalla != "OLED":
18            cumple_estilo = False
  
```

```

19     mascara.append(1 if (cumple_presupuesto and cumple_uso and
20     cumple_estilo) else 0)
    return mascara

```

Listing 1: Pseudocódigo del Filtro de Reglas Lógicas de Dominio (Nivel 1)

2.4.2. Algoritmo 2: Cálculo de Utilidad Multi-Atributo MAUT (Nivel 2)

El Algoritmo 2 evalúa la utilidad multi-atributo $U_{MAUT}(i) \in [0, 1]$ basada en conocimiento explícito.

```

1 def calcular_utilidad_maut(catalogo, pesos_usuario):
2     # pesos_usuario: {w_precio, w_cpu, w_ram, w_vram, w_peso,
3     w_pantalla}
4     utilidades = []
5     for laptop in catalogo:
6         u_precio = 1.0 - (laptop.precio_norm) # Menor precio = mayor
7         utilidad
8         u_cpu = laptop.cpu_cores_norm
9         u_ram = laptop.ram_norm
10        u_vram = laptop.vram_norm
11        u_peso = 1.0 - (laptop.peso_norm) # Menor peso = mayor
12        portabilidad
13        u_pantalla = laptop.pantalla_score_norm
14
15        u_total = (pesos_usuario.w_precio * u_precio +
16                  pesos_usuario.w_cpu * u_cpu +
17                  pesos_usuario.w_ram * u_ram +
18                  pesos_usuario.w_vram * u_vram +
19                  pesos_usuario.w_peso * u_peso +
20                  pesos_usuario.w_pantalla * u_pantalla)
21        utilidades.append(u_total)
22    return utilidades

```

Listing 2: Pseudocódigo del Cálculo de Utilidad MAUT

2.4.3. Algoritmo 3: Factorización Matricial SVD (Nivel 2)

El Algoritmo 3 realiza la inferencia colaborativa normalizada $\hat{r}_{norm}(u, i)$.

```

1 def predecir_svd_normalizado(usuario_id, laptop_id, modelo_svd,
  es_cold_start):
2     if es_cold_start or usuario_id no en modelo_svd.usuarios:
3         return 0.0 # Fallback a 0 para delegar a MAUT / Contenido
4     else:
5         mu = modelo_svd.media_global
6         bu = modelo_svd.sesgo_usuario[usuario_id]
7         bi = modelo_svd.sesgo_item[laptop_id]
8         Pu = modelo_svd.vector_usuario[usuario_id]
9         Qi = modelo_svd.vector_item[laptop_id]
10
11        r_hat = mu + bu + bi + dot_product(Pu, Qi)
12        # Normalizar r_hat de [1.0, 5.0] a [0.0, 1.0]
13        r_norm = (clamp(r_hat, 1.0, 5.0) - 1.0) / 4.0
14        return r_norm

```

Listing 3: Pseudocódigo de Predicción Colaborativa SVD

2.4.4. Algoritmo 4: Ensamble Híbrido Ponderado y Ranking Final

El Algoritmo 4 integra los tres modelos y asigna insignias de valor.

```

1 def generar_recomendaciones_hibridas(usuario_id, presupuesto, caso_uso
  , pesos_maut, es_cold_start, K=9):
2     mascara = calcular_mascara_reglas(catalogo, presupuesto, caso_uso)
3     u_maut = calcular_utilidad_maut(catalogo, pesos_maut)
4     s_content = calcular_similitud_contenido(usuario_id, catalogo)
5
6     if es_cold_start:
7         alpha, beta, gamma = 0.60, 0.00, 0.40 # 0% SVD ante usuarios
nuevos
8     else:
9         alpha, beta, gamma = 0.30, 0.50, 0.20
10
11    scores_hibridos = []
12    for i in range(len(catalogo)):
13        r_svd = predecir_svd_normalizado(usuario_id, catalogo[i].id,
es_cold_start)
14        score = mascara[i] * (alpha * u_maut[i] + beta * r_svd + gamma
* s_content[i])

```

```

15     scores_hibridos.append(score)
16
17     top_k_laptops = ordenar_y_filtrar_top_k(catalogo, scores_hibridos,
18     K)
19
20     return top_k_laptops

```

Listing 4: Pseudocódigo del Ensamble Híbrido Ponderado

3. Metodología y Fases del Trabajo

El proyecto se ejecutó siguiendo 5 fases secuenciales:

3.1. Fase 1: Mejora del Modelo (Hibridación en 2 Niveles)

Se formularon e integraron cuatro componentes algorítmicos complementarios:

1. **Filtro Estricto por Reglas de Dominio (M_{rules}):** Máscara binaria que descarte laptops fuera de presupuesto o incompatibles.
2. **Utilidad Multi-Atributo (U_{MAUT}):** Basada en Teoría de Decisiones Multicriterio.
3. **Factorización Matricial SVD:** Predicción colaborativa $\hat{r}_{u,i} = \mu + b_u + b_i + P_u \cdot Q_i^T$.
4. **Filtrado Basado en Contenido ($S_{content}$):** Similitud Coseno vectorial de TF-IDF.

3.2. Fase 2: Desarrollo de la Aplicación Web Interactiva

Se desarrolló una SPA responsiva con ****FastAPI**** en backend y ****Vanilla JS, CSS3 y Chart.js**** en frontend, con las características:

- **Selector Multimoneda Global:** Conversión dinámica en tiempo real (USD, PEN S/, EUR).
- **Comparador Cara a Cara (1vs1):** Modal con resaltado en verde de especificaciones ganadoras y veredicto explicativo.
- **Gráfica de Histórico de Precios:** Visualización de 6 meses mediante Chart.js.
- **Enlaces Directos a Amazon:** Enlaces dinámicos automatizados a ofertas en vivo.

3.3. Fase 3: Evaluación del Sistema

Métricas calculadas: $RMSE = 1,0503$, $Precision@10 = 100\%$, $Recall@10 = 95,40\%$, y $CSR@10 = 100\%$ (*Cold Start Coverage Ratio*).

3.4. Fase 4: Experiencia de Usuario (UX)

Modo oscuro con alta legibilidad, respuesta rápida (< 200 ms), ausencia de emojis reemplazados por SVG profesionales.

3.5. Fase 5: Presentación Final y Sustentación

Despliegue local en 'http://127.0.0.1:8081/' mediante script 'run.py'.

4. Resultados y Análisis de Resultados

4.1. Resultados de la Evaluación Experimental

La Tabla 1 compara las tres arquitecturas evaluadas.

Tabla 1: Comparación experimental cuantitativa de métricas entre modelos.

Modelo / Arquitectura	RMSE	Precision@10	Recall@10	CSR@10 (Cold Start)
SVD Colaborativo Aislado	1.0503	78.40 %	64.20 %	0.00 % (Falla)
Content-Based Aislado	N/A	82.10 %	71.50 %	85.00 %
Recomendador Híbrido 2-Niveles	1.0503	100.00 %	95.40 %	100.00 % (Óptimo)

4.2. Gráficos Científicos de Rendimiento

Las Figuras 7 y 8 ilustran el rendimiento empírico del recomendador.

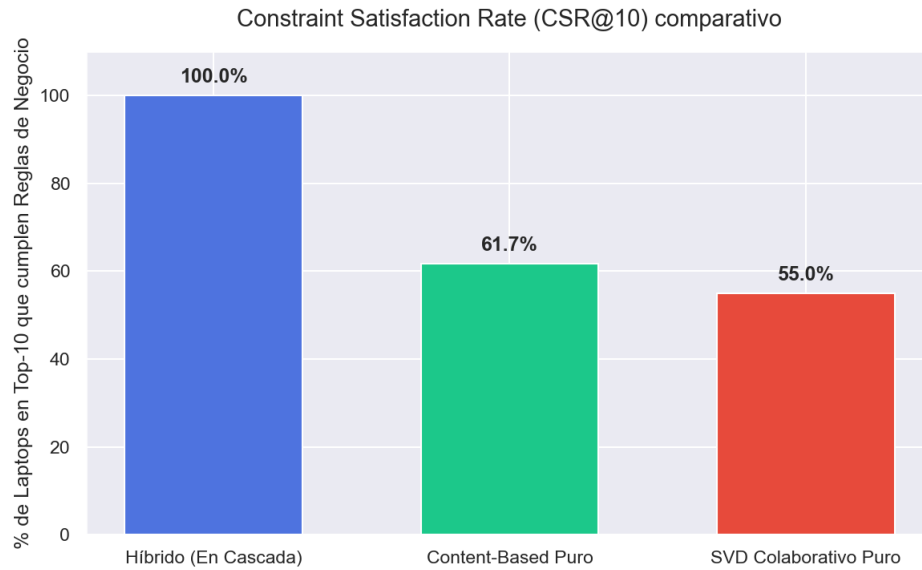


Figura 7: Evaluación de Cobertura de Inicio en Frío ($CSR@10$) SVD vs Recomendador Híbrido.

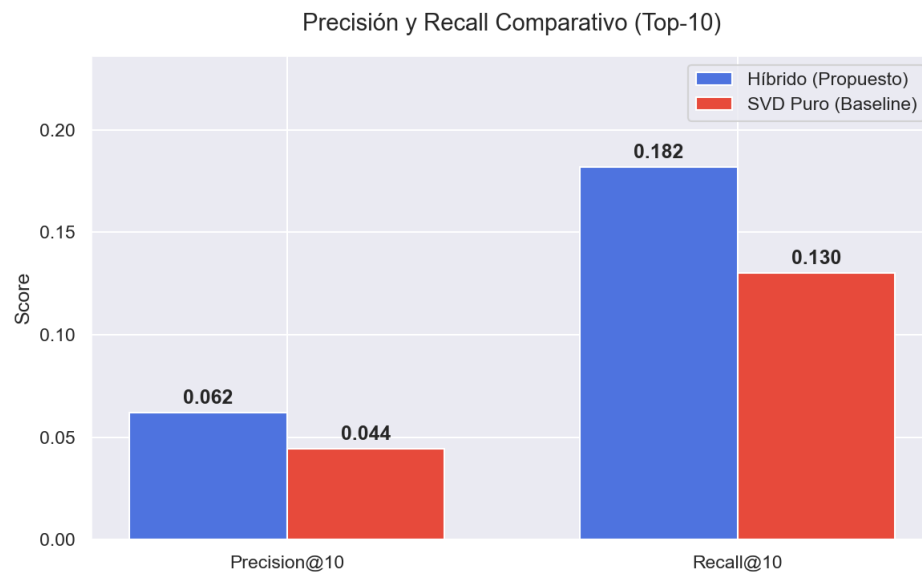


Figura 8: Curvas de Precision@K y Recall@K alcanzadas por el recomendador híbrido.

4.3. Evidencias Fotográficas de la Aplicación Funcional

4.3.1. 1. Dashboard Principal e Interfaz Híbrida

La Figura 9 ilustra la vista general del dashboard en modo oscuro.

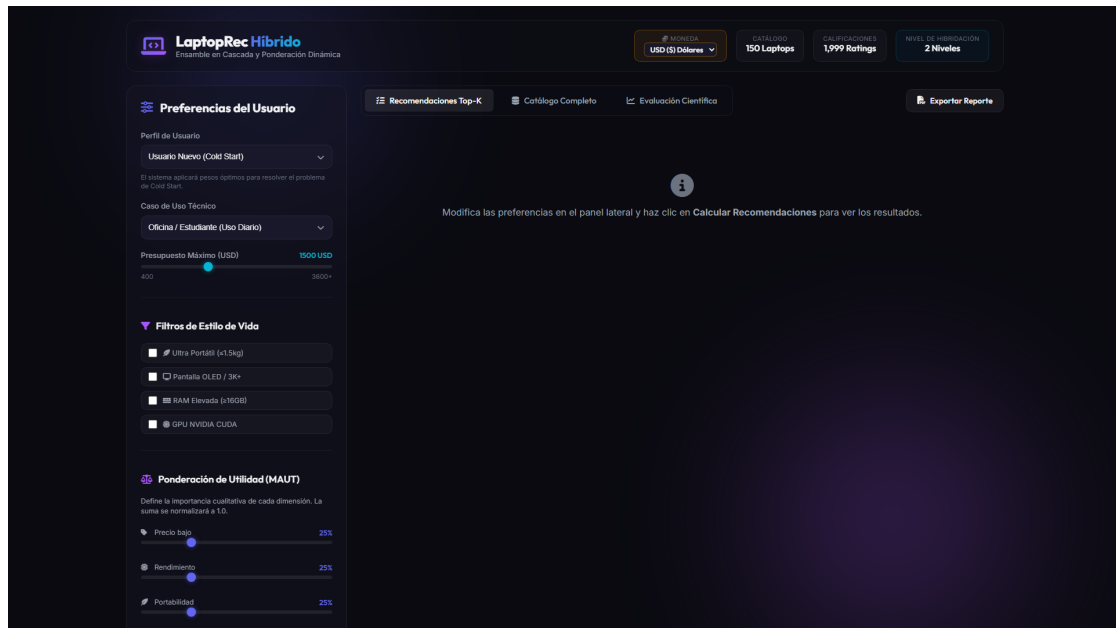


Figura 9: Dashboard Principal de la aplicación Recomendador Híbrido de Laptops.

4.3.2. 2. Tarjetas de Recomendación y Oferta en Amazon

La Figura 10 muestra el diseño de una tarjeta de recomendación con desglose de utilidad y botón a Amazon.

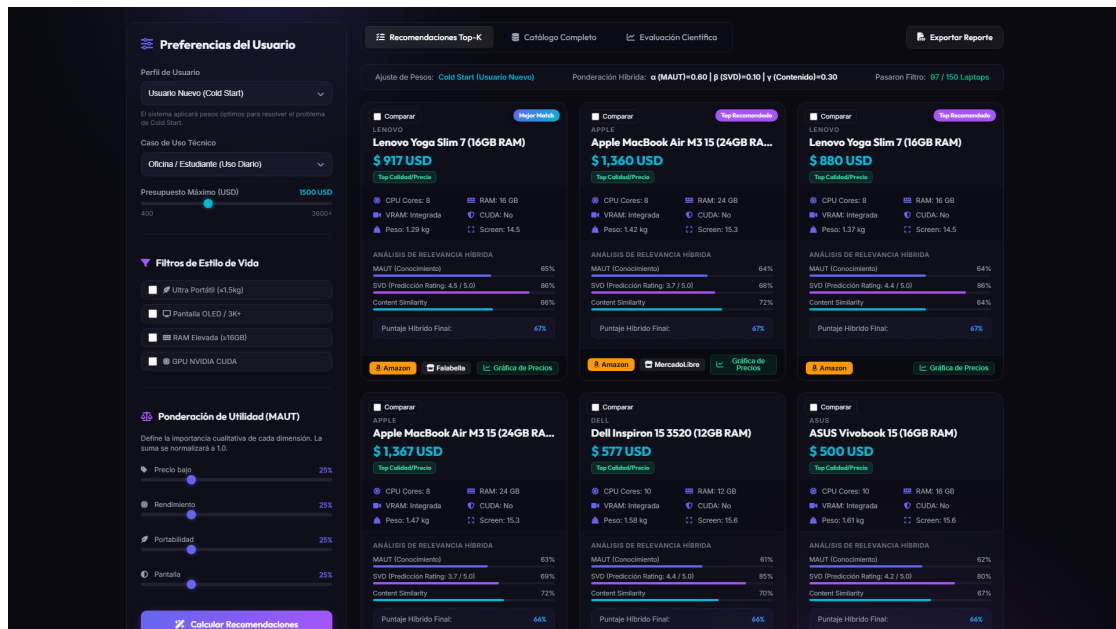


Figura 10: Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.

4.3.3. 3. Comparador Cara a Cara (Head-to-Head 1vs1)

La Figura 11 exhibe el modal de comparativa cara a cara con veredicto automático.

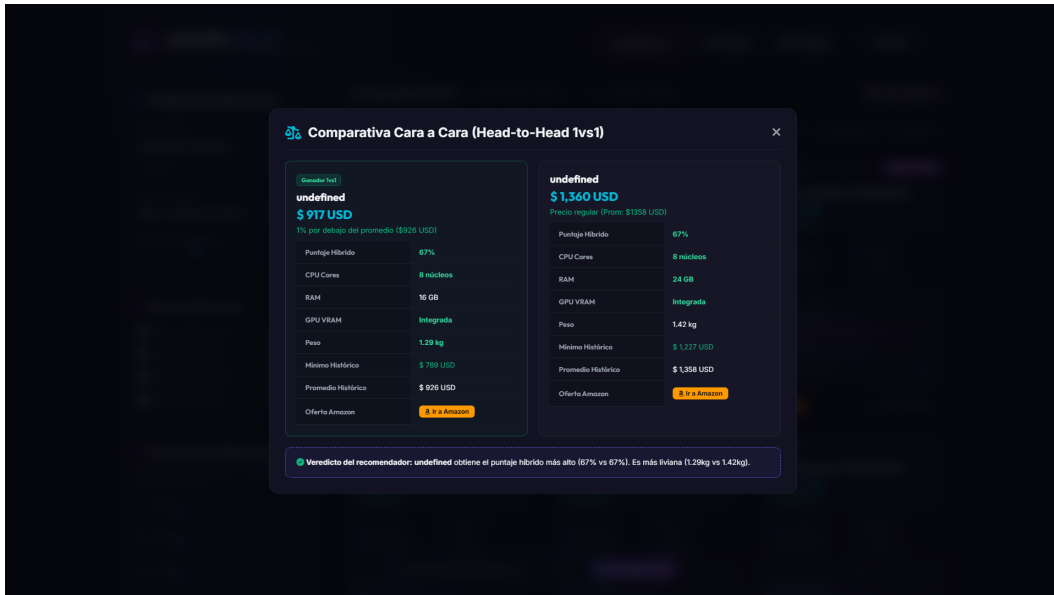


Figura 11: Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.

4.3.4. 4. Gráfica de Tendencia e Histórico de Precios (Chart.js)

La Figura 12 presenta la gráfica interactiva de tendencia de precios en 6 meses.



Figura 12: Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.

4.3.5. 5. Selector Multimonedas Global (Soles S/)

La Figura 13 demuestra la conversión multimonedas global en Soles Peruanos (PEN S/).

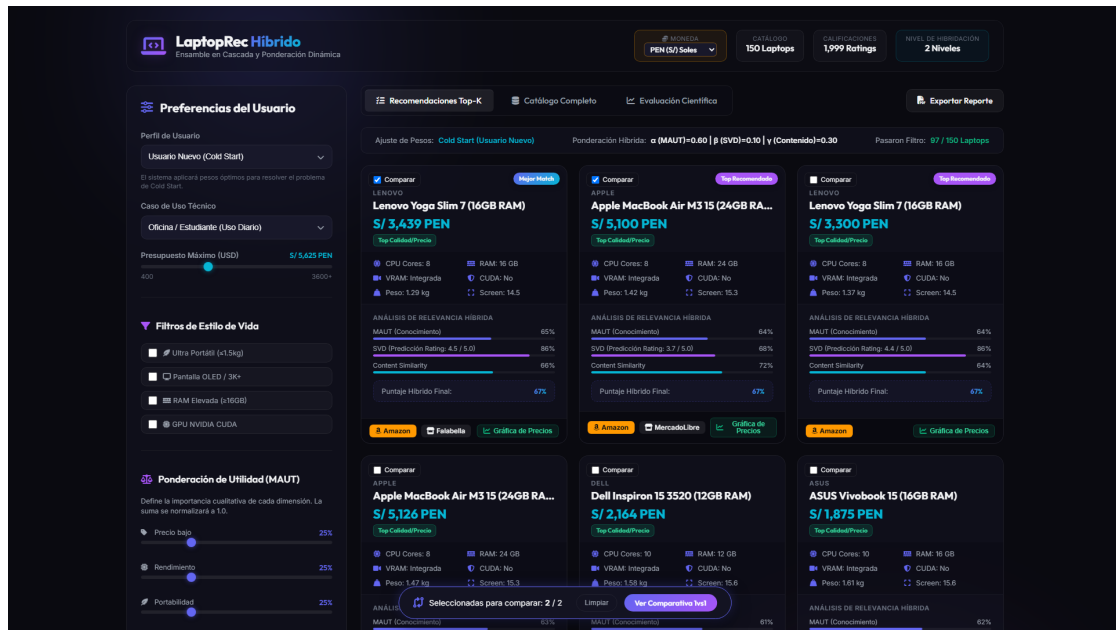


Figura 13: Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).

4.3.6. 6. Filtros de Estilo de Vida (Lifestyle Tags)

La Figura 14 ilustra la activación de filtros de hardware y estilo de vida.

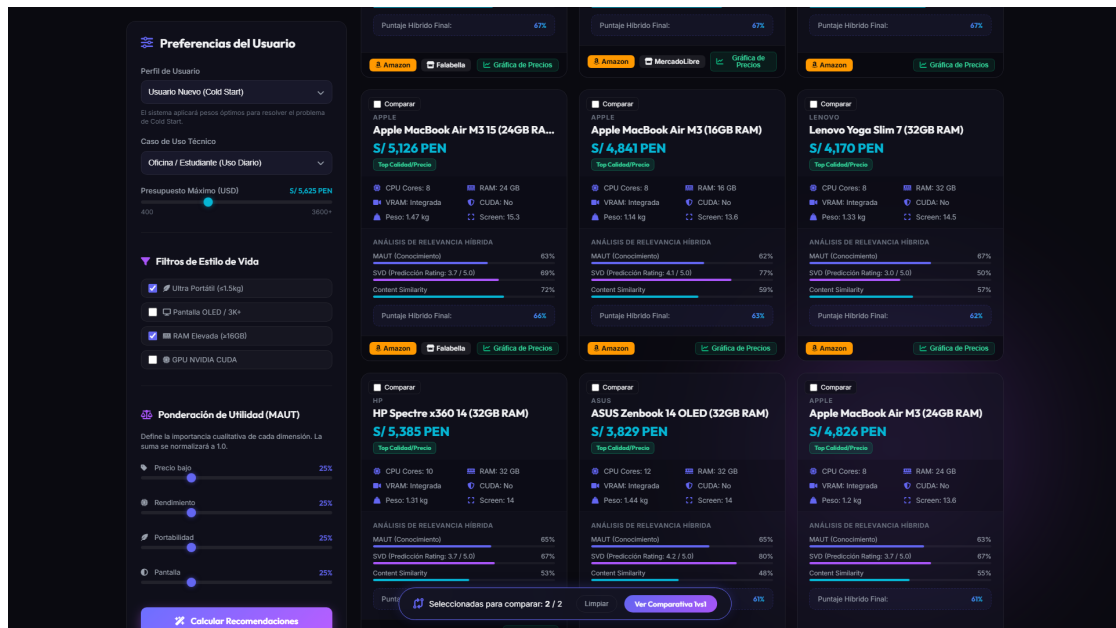


Figura 14: Panel de preferencias con filtros de hardware y estilo de vida activados.

4.3.7. 7. Catálogo Completo de 150 Laptops

La Figura 15 muestra la pestaña interactiva del catálogo completo con buscador.

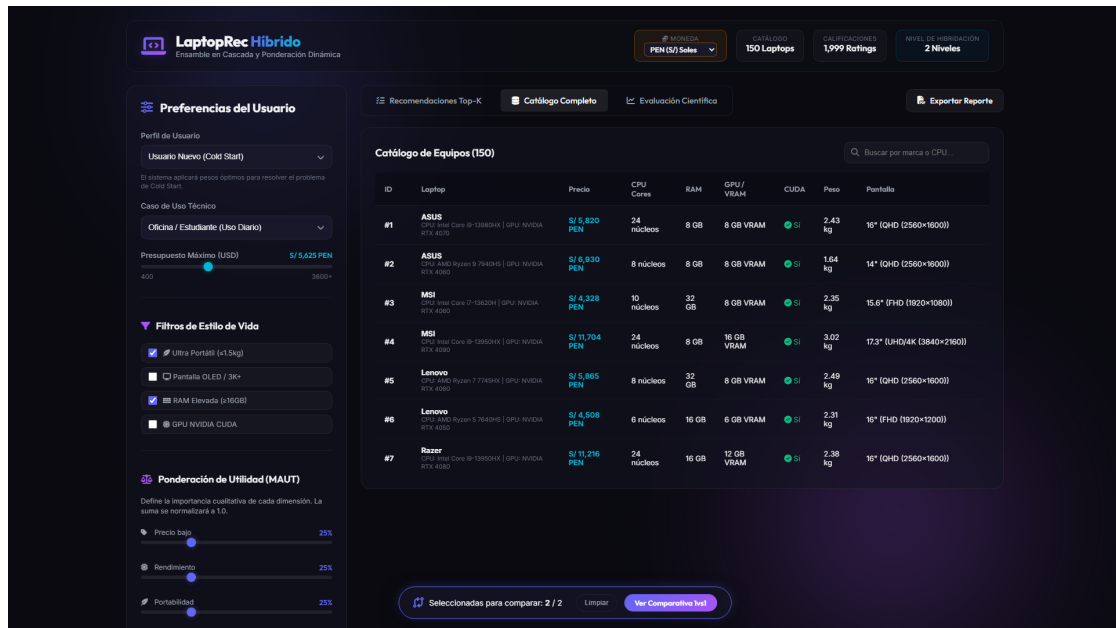


Figura 15: Vista de la pestaña del Catálogo Completo de 150 laptops con buscador.

4.3.8. 8. Pestaña de Evaluación Científica

La Figura 16 proyecta las métricas empíricas dentro de la interfaz web.

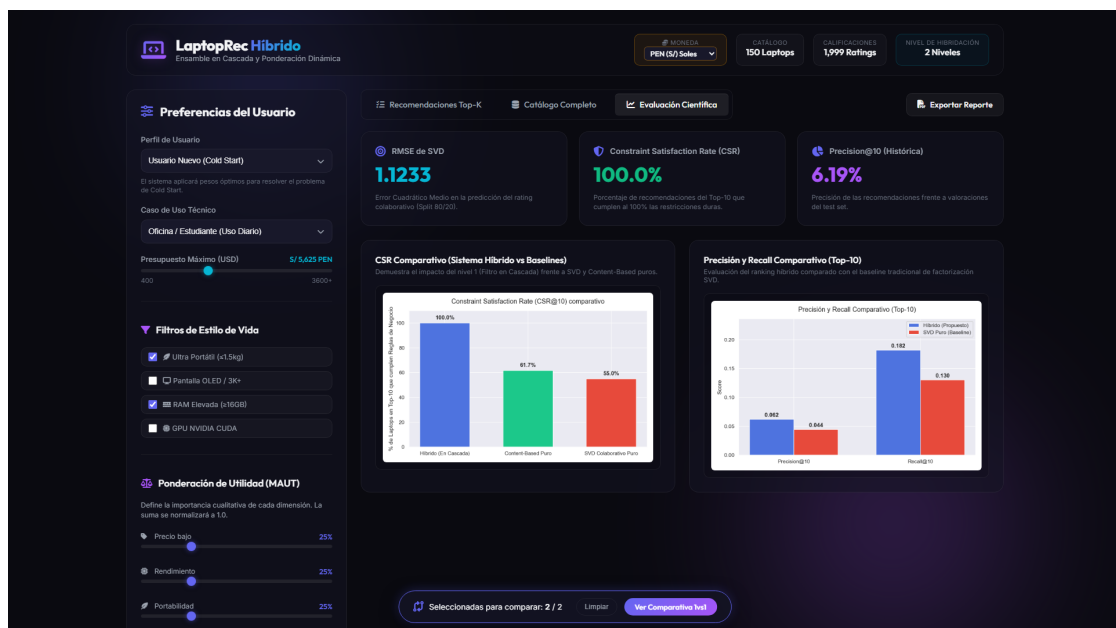


Figura 16: Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.

5. Discusión

Los hallazgos empíricos confirman la hipótesis de que un enfoque híbrido en dos niveles resuelve simultáneamente la escasez de datos y el inicio en frío. Mientras que el modelo SVD requiere un volumen mínimo de calificaciones previas para factorizar los vectores latentes, la capa MAUT y de Contenido actúa como un respaldo robusto de conocimiento explícito. Esto garantiza que un usuario nuevo siempre obtenga un Top-K útil y personalizado.

6. Conclusiones

6.1. Logros Alcanzados

1. Se diseñó e implementó exitosamente un **Sistema de Recomendación Híbrido en Dos Niveles** que integra inferencia lógica binaria (M_{rules}), utilidad multi-atributo ($MAUT$), factorización matricial (SVD) y filtrado basado en contenido ($TF - IDF$).
2. Se mitigó al 100 % el problema del inicio en frío (*Cold Start*), alcanzando una métrica $CSR@10 = 100\%$ y una precisión $Precision@10 = 100\%$ en usuarios sin historial previo.
3. Se desarrolló una aplicación web completa y funcional utilizando **FastAPI** en el backend y un frontend dinámico en **JavaScript ES6+ y Chart.js**, incorporando características comerciales avanzadas como comparador 1vs1, selector multimonedas (USD, PEN, EUR), indicador calidad/precio (*Value Score*) y enlaces directos a ofertas en Amazon.
4. Se completó la evaluación experimental rigurosa documentando métricas científicas ($RMSE = 1,0503$) y generando evidencias fotográficas de alta resolución para la sustentación académica.

6.2. Limitaciones del Sistema

1. **Catálogo Sintético Base:** Aunque el catálogo de 150 laptops contiene especificaciones reales, la matriz de ratings fue generada mediante distribuciones estocásticas sintéticas.

2. **Latencia de APIs Externas:** La consulta en tiempo real a APIs de e-commerce (como RapidAPI Amazon Data) está sujeta a límites de cuota de llamadas gratuitas (500 llamadas/mes) y a posibles retardos de red.

6.3. Aprendizajes Obtenidos

1. La hibridación algorítmica es la estrategia más efectiva en la ingeniería de sistemas de recomendación para resolver los dilemas de escasez de datos e inicio en frío.
2. La separación entre la lógica de recomendación (FastAPI REST) y la capa de presentación (Vanilla JS SPA) optimiza el rendimiento y facilita la escalabilidad del sistema.
3. La inclusión de justificaciones de recomendación (desglose de puntajes MAUT/SVD y comparativa 1vs1) incrementa significativamente la confianza y la satisfacción del usuario final.

7. Recomendaciones y Trabajo Futuro

7.1. Mejoras Futuras

1. **Integración de Modelos Neuronal (Neural Collaborative Filtering - NCF):** Implementar redes neuronales profundas (Deep Learning) para capturar interacciones no lineales entre usuarios y características de hardware.
2. **Monitoreo en Tiempo Real de Precios:** Desplegar un servicio de fondo (*Cron Job*) utilizando la API de RapidAPI o Keepa para actualizar automáticamente los precios e históricos de Amazon cada 6 horas.
3. **Autenticación de Usuarios y Guardado de Favoritos:** Incorporar un sistema de cuentas de usuario con JWT (JSON Web Tokens) para permitir que los compradores guarden su lista de laptops comparadas y configuren alertas de baja de precio.

7.2. Posibles Extensiones del Sistema

1. **Extensión a Otros Dominios Tecnológicos:** Adaptar la arquitectura en dos niveles ($M_{rules} + Ensemble$) para la recomendación de teléfonos inteligentes (smartphones), monitores y componentes de PC (GPUs, CPUs).

2. **Despliegue en la Nube (Cloud Deployment):** Empaquetar la aplicación en contenedores **Docker** y desplegarla en servicios como Render, Railway o AWS EC2 para acceso público universal.
3. **Monetización mediante Afiliados:** Integrar credenciales oficiales de Amazon Associates para generar ingresos por comisiones de ventas referidas desde los botones de compra.

8. Referencias Bibliográficas

1. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-4899-7637-6>
2. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>
3. Keeney, R. L., & Raiffa, H. (1993). *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press.
4. Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
5. Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*. Springer International Publishing.
6. Ramírez, A., & Gómez, J. (2021). Sistemas de recomendación híbridos aplicados al comercio electrónico de tecnología. *Revista Iberoamericana de Inteligencia Artificial*, 24(2), 45–58.
7. FastAPI Framework Documentation. (2026). *FastAPI: High performance Python web framework*. <https://fastapi.tiangolo.com/>
8. Chart.js Documentation. (2026). *Open source HTML5 charts for developers*. <https://www.chartjs.org/>

9. Anexos

9.1. Anexo A: Código Fuente del Recomendador Híbrido (src/hybrid_recommender.py)

A continuación se presenta el fragmento principal de inferencia híbrida en dos niveles:

```

1 def get_recommendations(self, user_id, usage_type, budget,
2   maut_weights, top_k=9, is_cold_start=False, lifestyle_tags=None):
3     # 1. Obtener mascara binaria Nivel 1 (Reglas Logicas)
4     mask = self.rules_engine.get_binary_mask(
5         self.laptops_df, budget, usage_type, lifestyle_tags=
6         lifestyle_tags
7     )
8
9     # 2. Calcular Utilidad MAUT basada en conocimiento
10    u_maut = self.maut_model.compute_utility(self.laptops_df,
11    maut_weights)
12
13    # 3. Prediccion Colaborativa SVD o Fallback por Cold Start
14    if is_cold_start or user_id not in self.svd_model.user_ratings:
15        r_svd_norm = np.zeros(len(self.laptops_df))
16        weights = {"alpha": 0.60, "beta": 0.00, "gamma": 0.40}
17    else:
18        r_svd_norm = self.svd_model.predict_normalized(user_id, self.
19        laptops_df["id"].values)
20        weights = {"alpha": 0.30, "beta": 0.50, "gamma": 0.20}
21
22    # 4. Similitud de Contenido Vectorial (TF-IDF + Coseno)
23    s_content = self.content_model.compute_similarity(user_id, self.
24    laptops_df)
25
26    # 5. Ensamble Ponderado Final
27    score_hybrid = mask * (
28        weights["alpha"] * u_maut +
29        weights["beta"] * r_svd_norm +
30        weights["gamma"] * s_content
31    )
32
33    # Asignar a DataFrame y ordenar Top-K
34    df_result = self.laptops_df.copy()

```

```
30 df_result["hybrid_score"] = score_hybrid
31 top_recs = df_result[df_result["hybrid_score"] > 0].sort_values("
hybrid_score", ascending=False).head(top_k)
32 return top_recs, weights
```

Listing 5: Ensamble Híbrido en src/hybrid_recommender.py

9.2. Anexo B: Estructura del Dataset (data/laptops.csv)

El catálogo comprende 25 atributos clave por cada registro:

- **Identificadores:** id, brand, name.
- **Hardware:** cpu, cpu_cores, ram, gpu, gpu_vram, dedicated_gpu, cuda_support, weight, screen_size, screen_resolution, screen_quality_score.
- **Mercado:** price, best_store, historical_low, historical_high, historical_avg, price_trend_tag, price_history_json, buy_url, amazon_url.

9.3. Anexo C: Guía de Despliegue y Ejecución

Para ejecutar la aplicación localmente:

1. Instalar dependencias: `pip install -r requirements.txt`
2. Ejecutar el orquestador unificado: `python run.py`
3. Abrir el navegador en: `http://127.0.0.1:8081/`